Programming Language 1
Python

Dr. Samera Obaid Barraood

IF ELSE STATEMENT IN PYTHON

AGENDA

- SIMPLE IF ELSE STATEMENT
- CONDITIONAL TESTS
- AND, OR OPERATOR
- ADVANCED IF ELSE STATEMENTS

EXAMPLE

- Example 1
  - number = 2
  - If number is odd, print “odd number”
  - If number is even, print “even number”
- Example 2
  - name = ‘Shivam’
  - If name is ‘Shivam’, print it in capital letters
  - Otherwise, print as it is

CONDITIONAL TESTS

Conditional Test

- If test is true, execute the code under if statement
- If test is false, ignore the code under if statement and move forward

CHECK FOR EQUALITY

```
>>> name = 'shivam'

>>> name == 'shivam'

True

>>> name = 'Anil'

>>> name == 'Shivam'

False
```

```
>>> name = 'shivam'

>>> name == 'Shivam'

False
```

CHECK FOR EQUALITY (Cont)

- Equality operator returns True if value on left and right side matches. Otherwise, False.
- Assignment operator(=) vs Equality operator(==)
- Assignment -> Storing a value in a variable
- Equality operator -> Are the values equal on both the sides ?

IGNORING THE CASE WHEN CHECKING FOR EQUALITY

- Gmail – freecodeschool vs FreeCodeSchool (@gmail.com)

```
names = ['Shivam', 'Anil', 'Navneet']

for name in names:

    if name.lower() == 'shivam':

        print(name.upper())

    else:

        print(name)
```

CHECKING FOR INEQUALITY

```
>>> name = 'shivam'

>>> name != 'shivam'

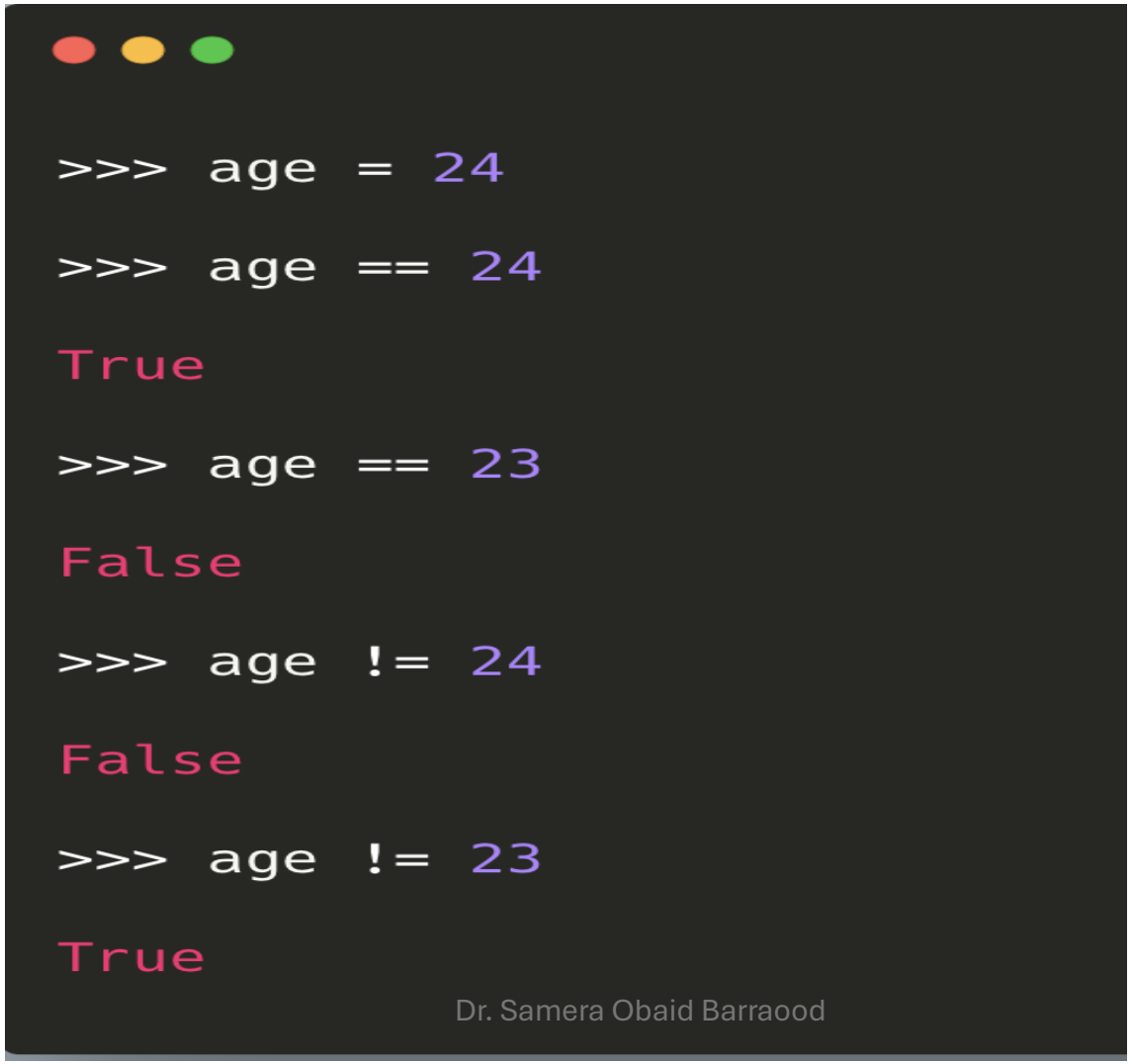
False

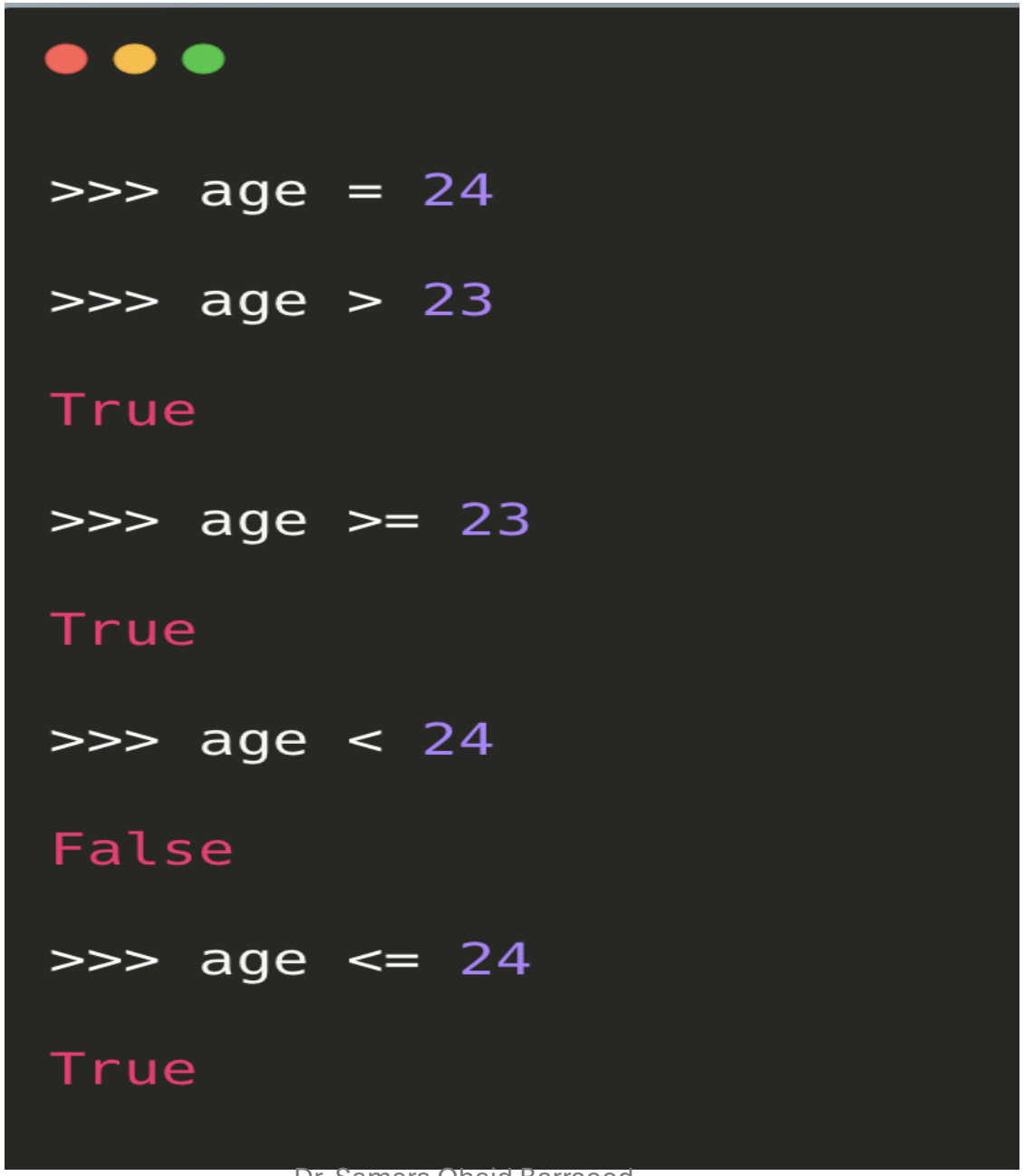
>>> name != 'Shivam'

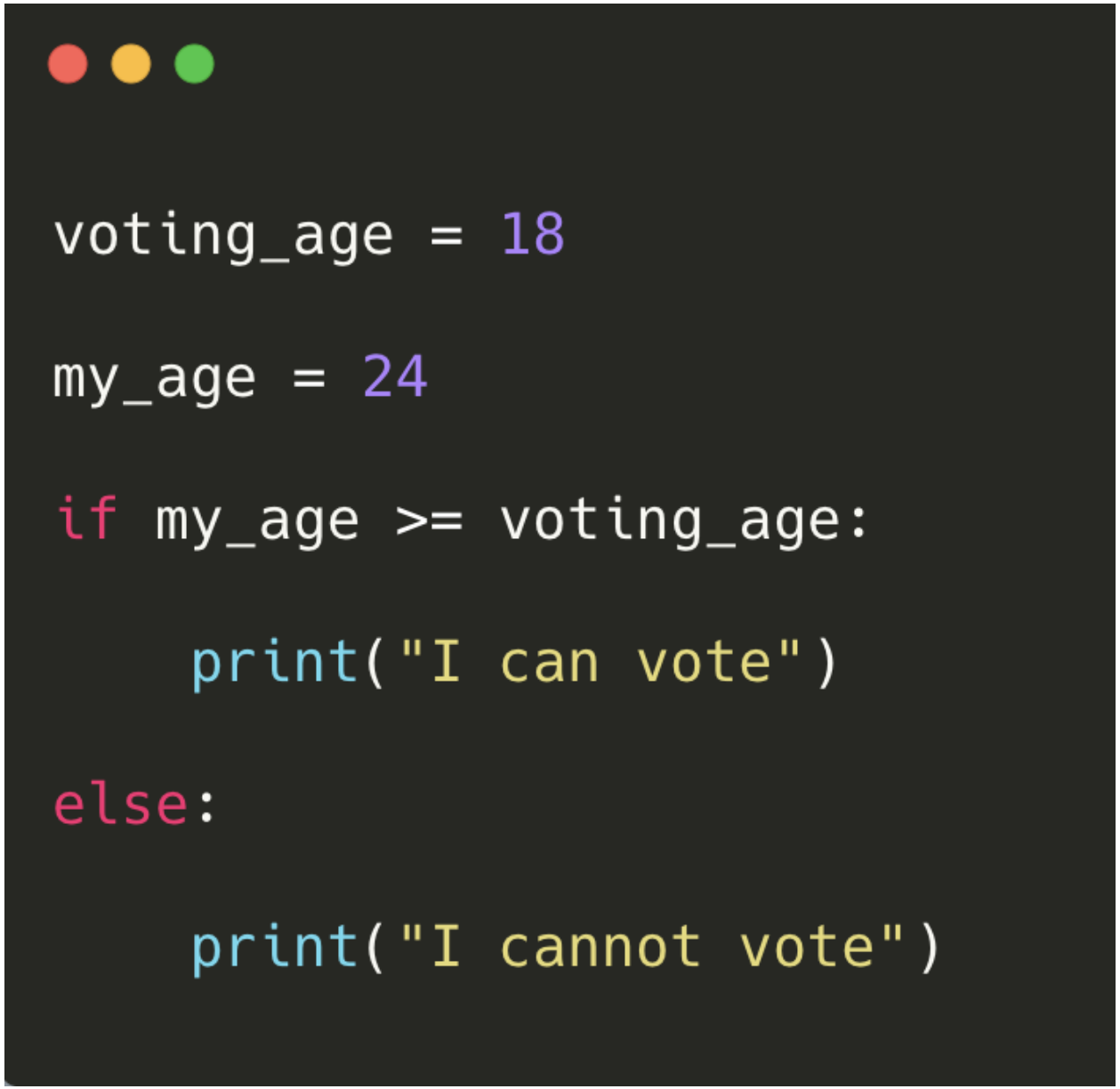
True
```

- If two values doesn’t match, True
- If two value matches, False

NUMERICAL COMPARISONS




>>> age = 24
>>> age > 23
True
>>> age >= 23
True
>>> age < 24
False
>>> age <= 24
True



```
voting_age = 18

my_age = 24

if my_age >= voting_age:

    print("I can vote")

else:

    print("I cannot vote")
```

CHECKING MULTIPLE CONDITIONS

- you might need two conditions to be True to take an action
- you might be satisfied with just one condition being True
- Keywords – and, or

USING AND TO CHECK MULTIPLE CONDITIONS

- Multiple tests
- If all test passes, return True
- If even one of the test fails, return False
- Checking happens in order
- If one test returns False, further checking doesn't happen

```
>>> age = 18

>>> age >= 10 and age <= 20

True

>>> age >= 20 and age <= 30

False
```

- More than 2 tests are also possible

using parentheses

```
>>> age = 18

>>> (age >= 10) and (age <= 20)

True

>>> (age >= 20) and (age <= 30)

False
```

```
>>> age = 18

>>> weight = 50

>>> age > 15 and weight >= 40 and weight <= 50

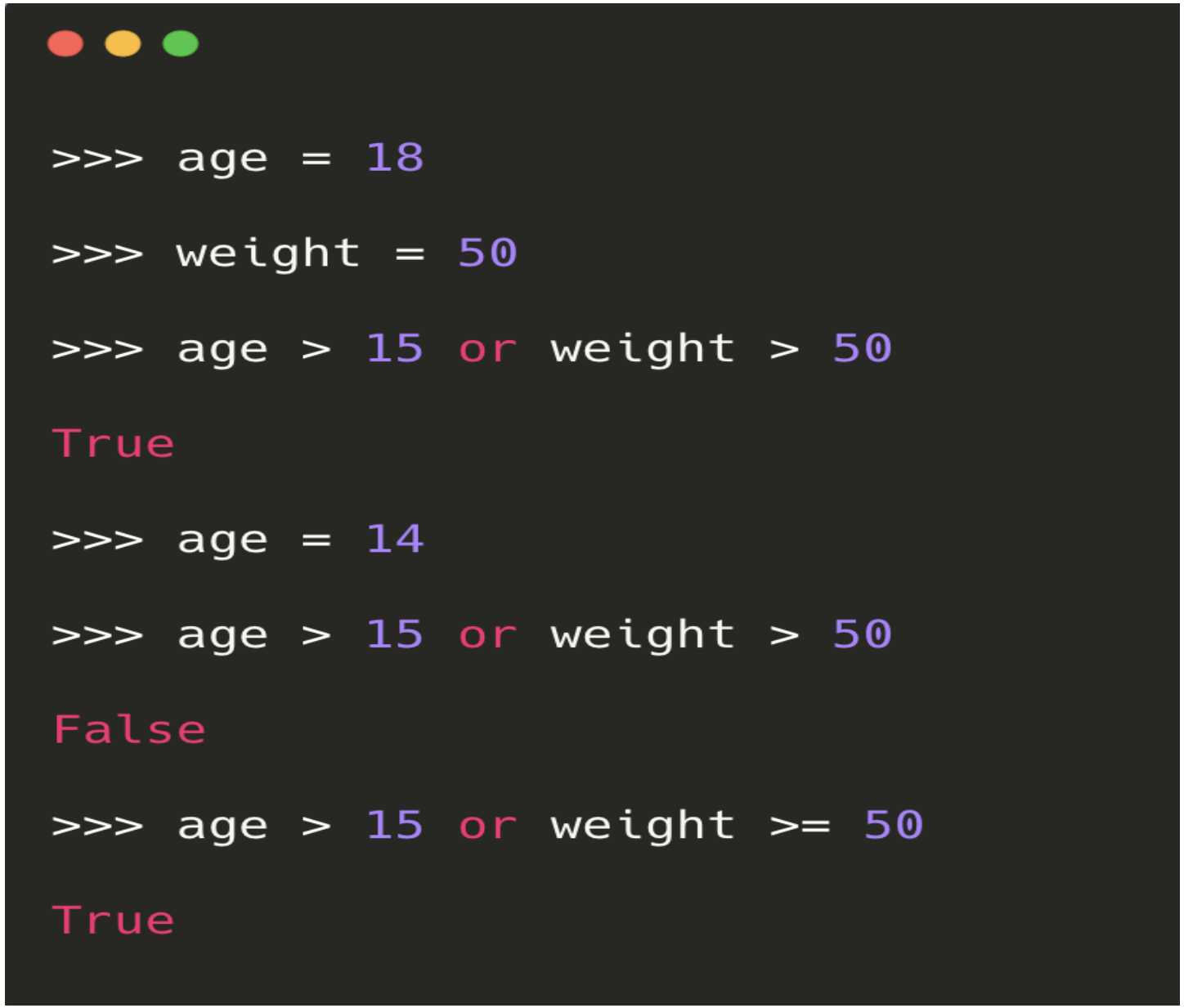
True

>>> age > 15 and (weight >= 40 and weight <= 50)

True
```

USING OR TO CHECK MULTIPLE CONDITIONS

- Multiple tests
- If any one of the test passes, return True
- Go in order and stop when any test passes
- If all the test fails, return False


>>> age = 18
>>> weight = 50
>>> age > 15 or weight > 50
True
>>> age = 14
>>> age > 15 or weight > 50
False
>>> age > 15 or weight >= 50
True

CHECKING WHETHER A VALUS IS IN THE LIST

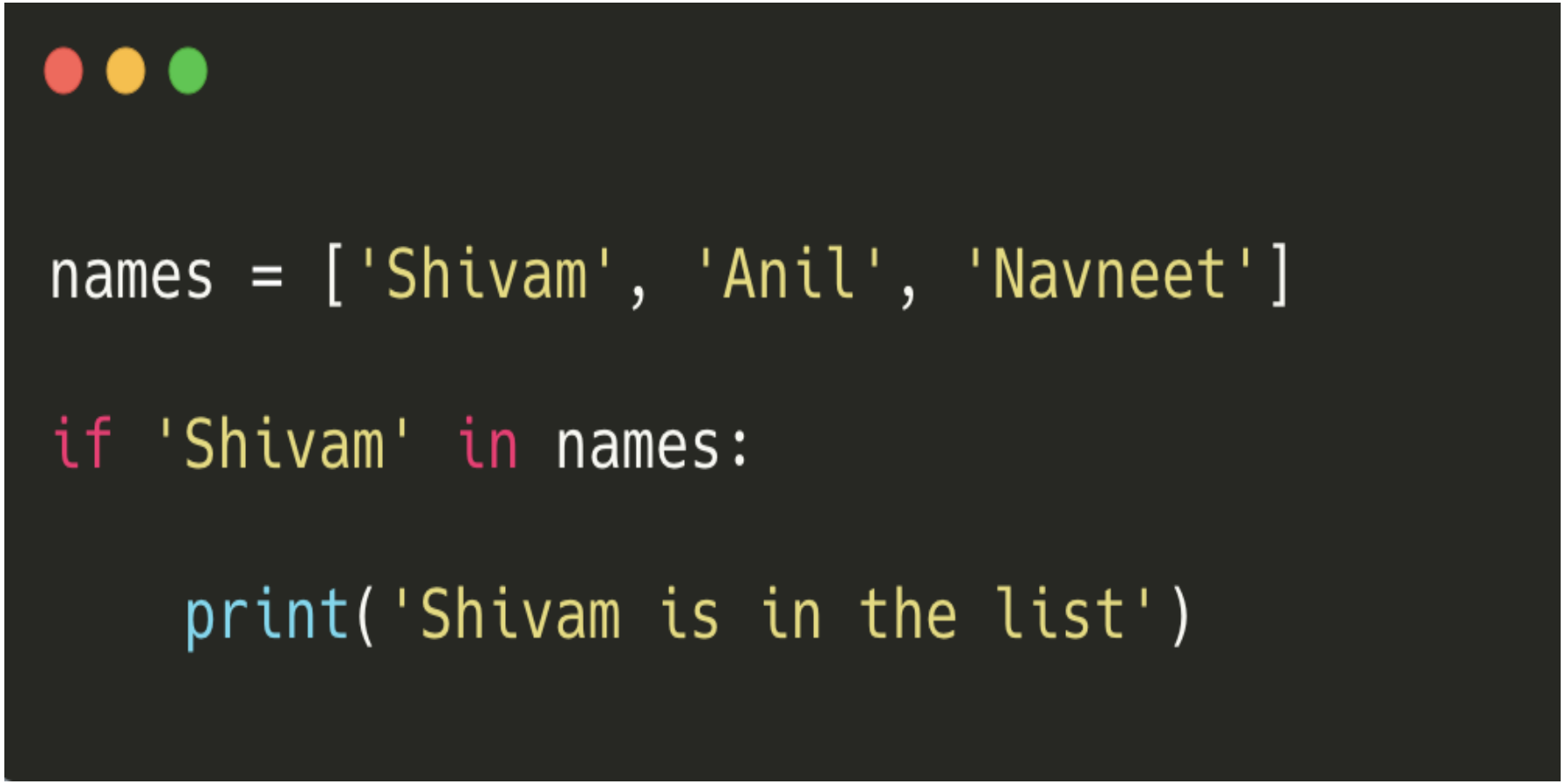
```
>>> names = ['Shivam', 'Anil', 'Navneet']

>>> 'Shivam' in names

True

>>> 'Shikha' in names

False
```



```
names = ['Shivam', 'Anil', 'Navneet']

if 'Shivam' in names:

    print('Shivam is in the list')
```

CHECKING WHETHER A VALUE IS NOT IN THE LIST

```
>>> names = ['Shivam', 'Anil', 'Navneet']

>>> 'Shivam' not in names

False

>>> 'Shikha' not in names

True
```

BOOLEAN EXPRESSIONS

- Another name for conditional statements
- A Boolean value is either True or False

```
can_edit = True

if can_edit:

    print("You can edit")

else:

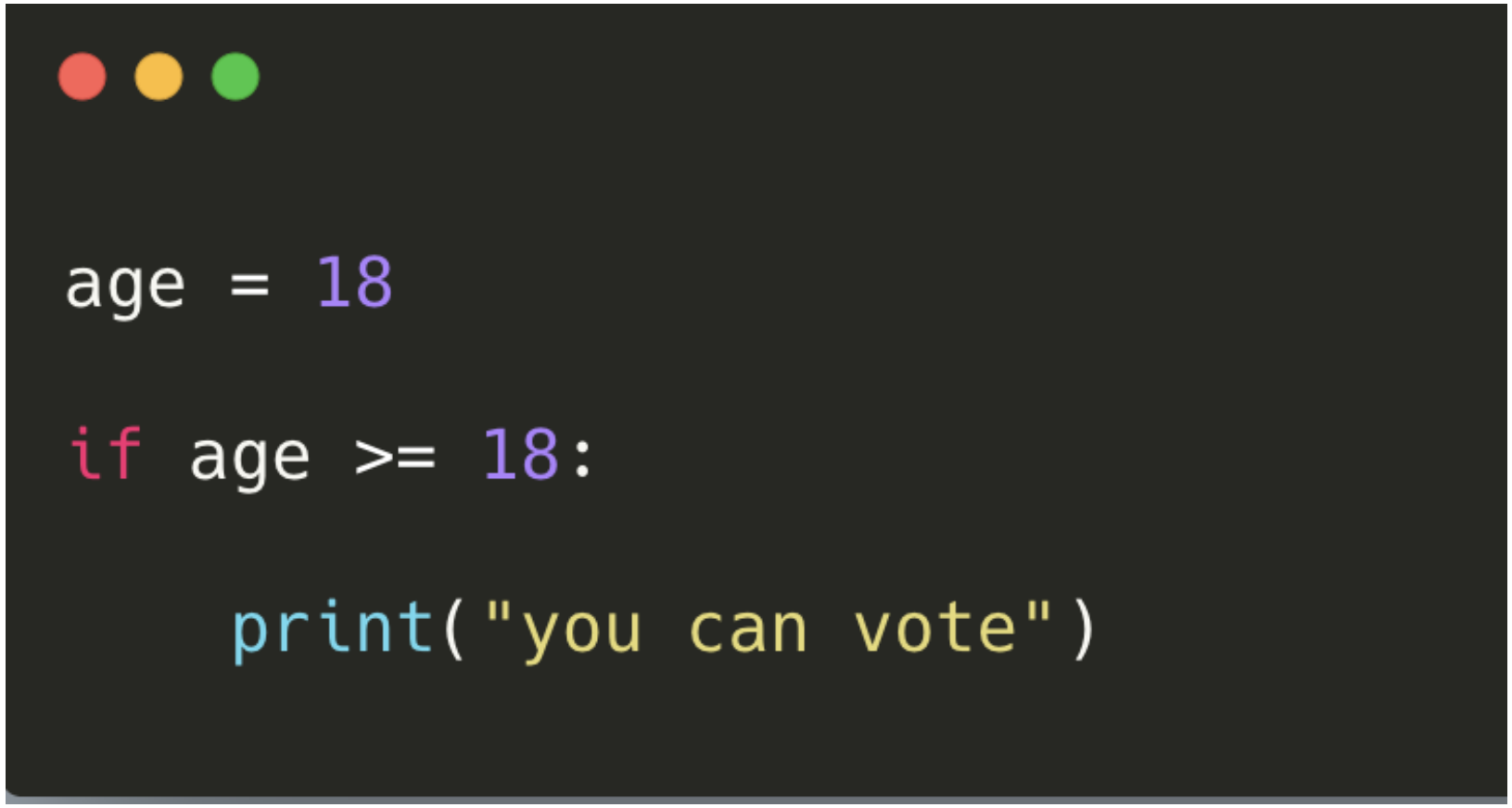
    print("you cannot edit")
```

IF STATEMENTS

SIMPLE IF STATEMENTS

```
if conditional_test:
    do something
```

You can put any conditional test in the first line and just about any action in the indented block following the test. If the conditional test evaluates to `True`, Python executes the code following the `if` statement. If the test evaluates to `False`, Python ignores the code following the `if` statement.


age = 18
if age >= 18:
print("you can vote")

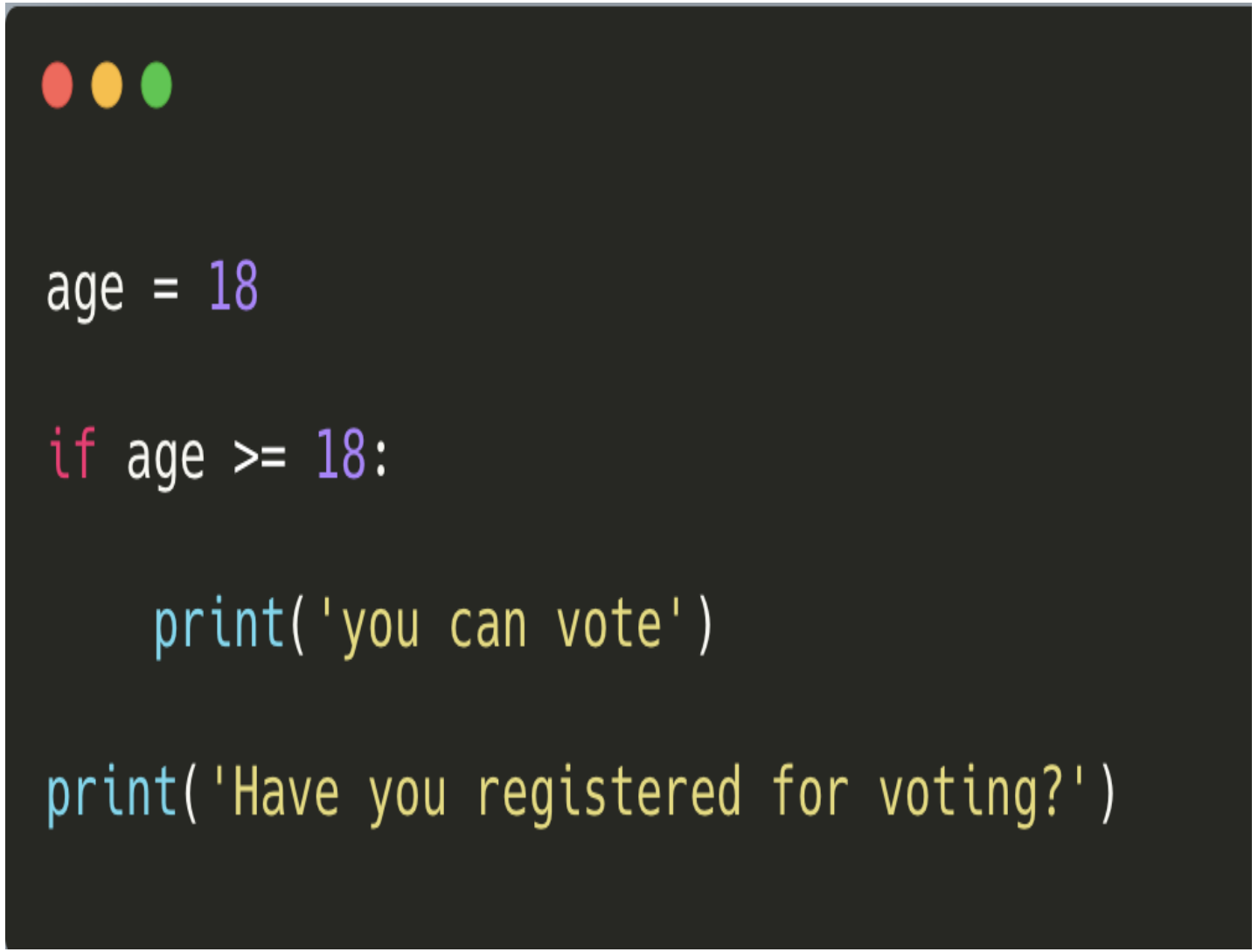
Indentation plays the same role in `if` statements as it did in `for` loops. All indented lines after an `if` statement will be executed if the test passes, and the entire block of indented lines will be ignored if the test does not pass.

```
age = 18

if age >= 18:

    print('you can vote')

    print('Have you registered for voting?')
```



```
age = 18

if age >= 18:

    print('you can vote')

print('Have you registered for voting?')
```

IF-ELSE STATEMENTS

Often, you'll want to take one action when a conditional test passes and a different action in all other cases. Python's `if-else` syntax makes this possible. An `if-else` block is similar to a simple `if` statement, but the `else` statement allows you to define an action or set of actions that are executed when the conditional test fails.

```python
age = 17

if age >= 18:

    print('you can vote')

    print('Have you registered for voting?')

else:

    print('sorry, you cannot vote')

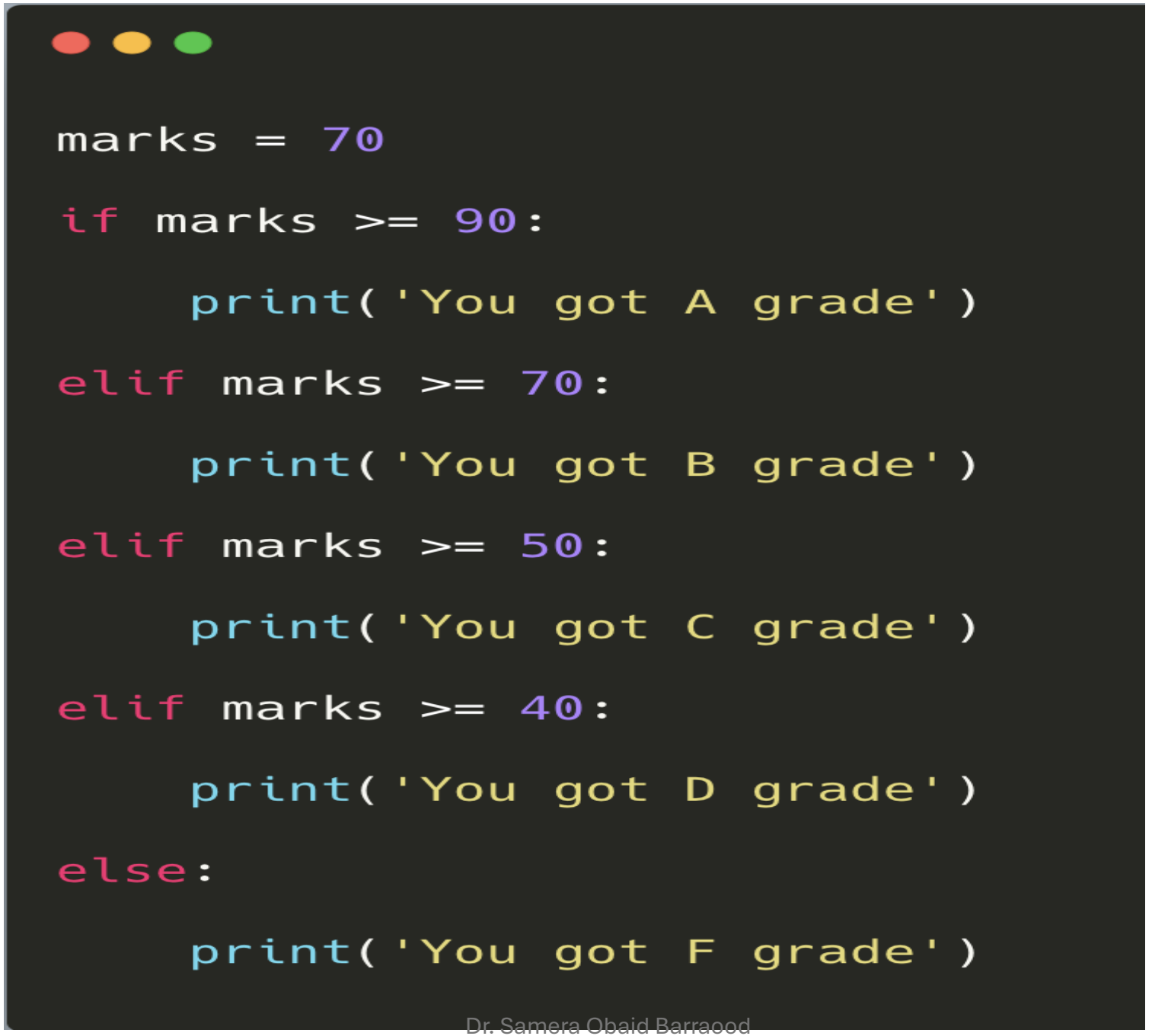
    print('do register to vote as soon you turn 18')
```

IF-ELIF-ELSE CHAIN

Often, you'll need to test more than two possible situations, and to evaluate these you can use Python's `if-elif-else` syntax. Python executes only one block in an `if-elif-else` chain. It runs each conditional test in order until one passes. When a test passes, the code following that test is executed and Python skips the rest of the tests.

GRADING

- marks >= 90 and marks <= 100 – A
- marks >= 70 and marks < 90 – B
- marks >= 50 and marks < 70 – C
- marks >= 40 and marks < 50 – D
- Otherwise, F



```
marks = 70
if marks >= 90:
    print('You got A grade')
elif marks >= 70:
    print('You got B grade')
elif marks >= 50:
    print('You got C grade')
elif marks >= 40:
    print('You got D grade')
else:
    print('You got F grade')
```

OMITTING THE ELSE BLOCK

```
marks = 70
if marks >= 90:
    print('You got A grade')
elif marks >= 70:
    print('You got B grade')
elif marks >= 50:
    print('You got C grade')
elif marks >= 40:
    print('You got D grade')
elif marks < 40:
    print('You got F grade')
```

TESTING MULTIPLE CONDITIONS

```
names = ['Shivam', 'Anil', 'Navneet']

if 'Shivam' in names:

    print('Shivam name is present')

if 'Anil' in names:

    print('Anil name is present')
```

USING IF STATEMENTS WITH LISTS

- ODD EVEN EXAMPLE
- SEARCH A NAME EXAMPLE

CHECKING IF A LIST IS EMPTY

```
names = []

if names:

    print('the list is empty')

else:

    print('the list is not empty')
```

ASSIGNMENT

a = 2

b = 3

If a is greater than b, print “a > b”

If a is smaller than b, print “a < b”

If a is equal to b , print “a = b”

Note: a and b can take any values

Thank You

